Effective AI Usage

14.33 Research and Communication in Economics

Jon Cohen

September 28, 2026

Agenda

1. Agentic AI
2. Context management
3. Intellectual ownership
4. Verification
5. Permissions

Cheap execution makes judgment and verification more important

Cheaper with AI	More important with AI
Implement a known estimator	Choose a credible research design
Clean and restructure data	Identify high-value datasets
Write and debug code	Implement the right verification tests
Make figures and tables	Decide how to present the main idea efficiently

Good agent use applies established software practices

Software practice	Application to agents
Do not repeat yourself	Put recurring instructions in files and code
Documentation	Keep durable decisions and plans in the repository
Version control	Review each agent change as a commit
Testing	Protect checks and review changes to them
Least privilege	Give the agent only the access it needs

Recall purpose of this course's AI disclosure policy

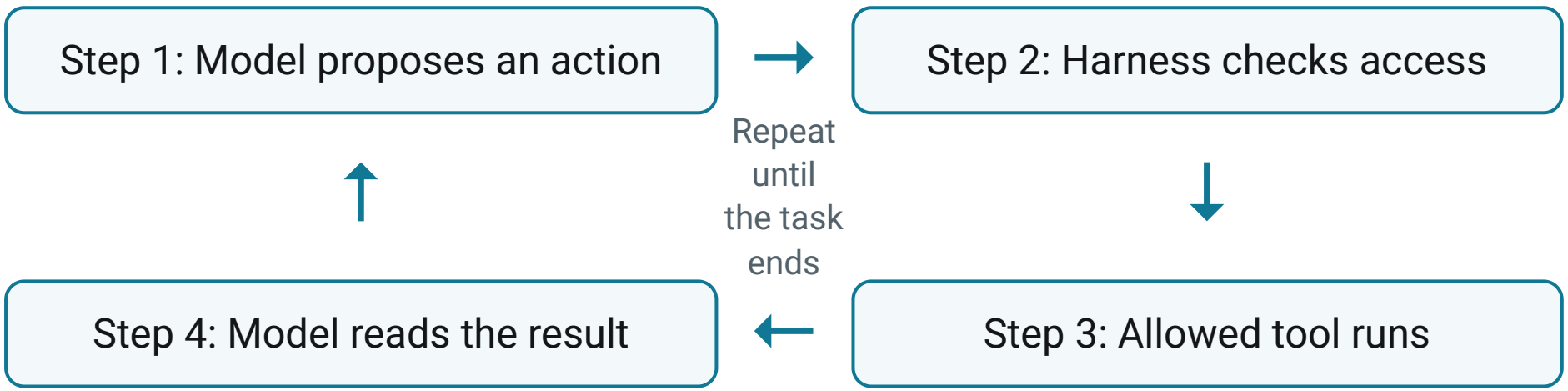
	Can help	**Can hurt**
Short-term	Iterate on code quickly	Follow a reassuring but unproductive approach
Long-term	Reveal tools and practices you did not know	Replace practice you need to learn implementation

1. Agentic AI

- An agent is a model that uses tools in a loop
- Agents fit long projects whose work lives in files
- Chat access to a model is not agent access

An agent is a model that uses tools in a loop

You set the task and constraints. The agent repeats this cycle:



The harness is the program, such as Codex or Claude Code. Tools include files, commands, and web access.

The loop lets an agent recover from errors

Task: “Get homeownership rates and average wages by age.”

1. Searches a familiar data site. The series it finds lack age detail.
2. Tries a government site. A request gets a 403 error.
3. Reads the error and changes the request header.
4. Downloads two tables, parses them, and joins them by age.

Exercise

1. What could still be wrong even if the code runs?
2. How could you set up the task better?

Terminology clarification on “agent”

Phrase	What it means here
Coding agent	The running tool, such as Codex or Claude Code
Custom agent or subagent	An optional second agent with a focused role or tools
`AGENTS.md`	Project instructions that a coding agent reads. The file does **not** create a custom agent

For projects, agentic AI is better than AI chatbots

	Web chat with uploads	**Agent in a project folder**
What it sees	Files that you upload to each chat	Files that it finds in the project
Where code runs	The provider’s environment	Your computer, with your data and software
Where work stays	The chat history	Files and commits

LLMs support many levels of autonomy

Level	You	The AI
Chat	Copy code in and out	Suggests text
Inline completion	Write code	Completes the line
Agentic editor	Describe a task	Edits files in the editor
Command-line agent	Describe a task	Reads, edits, runs, iterates
Autonomous agent	Set the goal and the environment	Works for hours without approval

Which level describes your most recent use?

MIT Agentic AI Infrastructure: Parley

Do not put sensitive or Institute data into public third-party models. Check MIT's data guidance before using any AI tool.

Route	What you can do
Web chat	Converse in a browser. No local coding-agent tools
API	Application programming interface. Call models from code or a coding agent. Pay per token

API suggestion: students get $10 of initial credit. Try a small batch of model-coded records, inspect labels and spend, then decide whether to scale.

Running example: public election reports in PDFs

- A government site links to hundreds of county election reports
- The goal is one row per county, year, contest, and candidate
- A coding agent can help build download and extraction scripts
- The researcher checks source coverage and vote totals

We will use this project to decide what belongs in a prompt, project instructions, a skill, or a tool setting.

Running example: does each election PDF need a model call?

Pipeline	Who handles each PDF?	What scales with the number of PDFs?
Local extraction	Saved text-extraction or optical character recognition (OCR) script	Computer time
Model extraction	Saved script calls a model API for each document or page	API usage cost

A subscription coding agent can **help build either script** and inspect sample election reports interactively. If the saved script itself calls a hosted model for each PDF, budget for a separate, metered API.

Terminology recap: agentic AI

Term	Meaning
Token	A unit of text, about 3/4 of a word
Context window	The maximum number of tokens that a model reads on one turn
Model	Reads the context window and predicts the next tokens
Tool	An action outside the model, such as reading a file or running a command
Harness	The program that gives a model tools and runs them, such as Claude Code, Codex CLI, or Gemini CLI
Agent	A model and a harness, working in a loop
Model Context Protocol (MCP)	A standard way to connect a harness to tools or data sources

Takeaways: agentic AI

- An agent is a model with tools that works in a loop
- It observes results, so it can recover from errors
- Its advantage is largest when the work persists in a project
- Browser chat alone is not agent access. An API can also power a coding agent

2. Context management

- The agent knows only what is in its context window
- Write project knowledge to plain-text files
- Do not repeat yourself: codify recurring instructions

What enters the context window on each turn

```
At session start:
  System instructions + project instructions (AGENTS.md or CLAUDE.md)
  Tool descriptions + brief names/descriptions of available skills

Conversation so far:
  Your prompts + agent replies + tool calls and their results

When relevant:
  Full SKILL.md and its supporting files, after the agent opens them

New prompt → model sees this context and chooses the next step
```

A heuristic: wrong, missing, or extra context lowers performance

$$\text{performance} \sim \frac{\text{correctness}^2 \times \text{completeness}}{\text{size}}$$

1. Wrong information does the most damage
2. Missing information makes the agent guess
3. Irrelevant information dilutes the relevant information

Source: HumanLayer, “Advanced context engineering for coding agents”

Keep each session to one task

Practice	Reason
Start a new session for each task	Old turns stop competing for attention
Rewind a wrong path instead of arguing	The wrong path leaves the window
Compact on purpose, with a focus	You choose what the summary keeps
Send searches to a subagent	Search results stay out of the main window

Carry your plan into a new session

1. Decide the goal and review the plan with the agent
2. Save your decisions and next step to `docs/progress.md`
3. Start a new session
4. Ask the agent to read that file and confirm its understanding

Plain-text files work well for both agents and git

Easy to diff and search	Needs extra work to inspect
Markdown, plain text	Word documents, slide decks
Scripts: `.py`, `.R`, `.sh`	Notebooks with saved outputs
Small `.csv`, `.tsv`	Excel workbooks, images of figures
Text-based PDFs	Scanned PDFs, images of tables

Example: Project directory with context files

```
election-project/
├── README.md                Project overview, directory structure, and how to run it
├── AGENTS.md                Project instructions, explained shortly
├── DECISIONS.md             Major research choices and reasons
├── docs/                    Source descriptions and methodological notes
│   ├── source-notes.md      URLs, formats, and source caveats
│   └── methodology-notes.md  Definitions and design choices
├── code/                    Download and extraction scripts
├── data/
│   ├── raw/                 Saved source PDFs
│   └── clean/               Generated analysis data
└── output/                  Figures and tables
```

Keep durable research decisions with the project

```
## 2026-09-20: Which election reports are authoritative?

Decision:    Use certified county canvass PDFs.
Alternative: Use election-night reports, which may change later.
Reason:      Certified reports are the final official totals.
Evidence:    docs/source-notes.md
Issue:       https://github.com/your-name/election-project/issues/23
```

Use GitHub issues for open questions and work that needs tracking. For a **major research decision**, put the implemented rule in code or tests and keep its rationale in project documentation. You do not need an issue for every line of code.

Choose the instruction by when it should apply

When should it apply?	Put it here
Just this task	Your prompt
Every session in this repo	`AGENTS.md` or `CLAUDE.md`
When a repeatable task comes up	`SKILL.md`

`AGENTS.md` or `CLAUDE.md`: short directions for every session

Save one at the project root, then commit it to Git.

```
# Project instructions
- Keep source PDFs in data/raw/ unchanged. Write derived data to data/clean/.
- Put each data transformation in code/ and use project-relative paths.
- Run `python -m pytest -q` before reporting a completed code change.
- Record durable source and schema decisions in DECISIONS.md.
```

This file guides the model. It does not set tool permissions.

Starting: create and refine `AGENTS.md`

Open the project in a command-line interface (CLI):

```
cd election-project
git status            # confirm the repository
codex                 # or launch claude
```

1. **Write a first version yourself.** Include only standing project facts: the directory map, exact test command, and raw-data rule.
2. **Ask the coding agent to audit it.** Have it check paths and commands against the repository. You accept or reject each suggested edit.
3. **Commit and revise it.** Add a rule after a repeated correction or a workflow change. Review the instruction-file diff.

The CLI starts the agent and `AGENTS.md` guides it.

`SKILL.md`: a recipe for a task you repeat

Once adapting the parser to another report layout becomes repeatable, save a recipe in a named project skill folder. Its instruction file is `SKILL.md`, singular.

```
---
name: add-report-layout
description: Use when adapting the election parser to a new PDF layout.
---
1. Inspect representative PDFs and record the layout in docs/source-notes.md.
2. Update the parser in code/ and add test fixtures for the new layout.
3. Run the extraction and tests from a clean checkout.
4. Report coverage gaps and rows needing a hand check.
```

The agent reads the full recipe when this task comes up. Do not make a skill for a task you will do only once.

Same Markdown, different jobs

	`AGENTS.md` / `CLAUDE.md`	`SKILL.md`
Question	How do we work in this repository?	How do we do this kind of task?
Content	Short, broadly useful rules and project facts	Steps, references, and scripts for one repeatable workflow
Loaded first	The project instructions	The skill's name and description
Election project	Keep raw PDFs unchanged. Save transformations as code.	When a new layout appears, update the parser and its tests.

The full skill procedure loads **when the task calls for it** (i.e., progressive disclosure).

Build project instructions as the work matures

Suggested workflow

1. Establish `AGENTS.md` with short election project instructions at the start.
2. Add a skill after a repeatable procedure emerges.
3. Revise both as the project matures.

The saved scripts and tests reproduce the data. These files guide the agent's work on them.

Exercise: what goes in `AGENTS.md` versus `SKILL.md`?

For each line, choose **project instructions**, **a skill**, or **somewhere else**. Explain your choice.

1. "Use project-relative paths in all scripts."
2. "Run `python -m pytest -q` before reporting a completed code change."
3. "When adding a county, check whether its PDFs have a text layer."
4. "For each new county, compare parsed row counts with source totals."
5. "Today, inspect just these three counties."
6. "Block the agent from writing to `data/raw/`."

Would you create the skill before you have processed your first county?

Terminology recap: context management

Term	Meaning	Claude Code, Codex, Gemini CLI
Instruction file	Short project directions read each session	`CLAUDE.md`, `AGENTS.md`, `GEMINI.md`
Draft an instruction file	Generate a first version, then edit it	`/init` in Codex or ask the agent
New session	Start with an empty window	`/clear`, `/new`, `/clear`
Compaction	Summarize the window to free space	`/compact`, `/compact`, `/compress`
Plan mode	Read and propose, but do not edit	`/plan` in all three
Skill	A reusable procedure read for a relevant task	Singular `SKILL.md`, listed by `/skills`

Takeaways: context management

- Each session starts empty. The agent knows only what is in its window.
- Keep each session focused. Compact or restart on purpose.
- Keep decisions and plans in plain-text files in the repository
- Use project instructions for every-session rules and skills for repeated tasks
- Put enforceable checks in code or tool settings, not only in instructions

3. Intellectual ownership

- You are accountable for the result
- Make the key decisions yourself, before the agent does
- Plan first, then work in small steps
- Recall purpose of this course's individualized final exam

You remain accountable for work that the agent does

IBM training manual (1979):

"A computer can never be held accountable. Therefore a computer must never make a management decision."

Garicano, Li, and Wu, *Messy Jobs* (2026):

AI can take on well-specified tasks. People still supply judgment and accept responsibility for unpredictable outcomes.

Good prompting is project management

- State the goal and what a usable result looks like
- Give facts, constraints, and source material the agent cannot infer
- Ask it to surface choices and uncertainties before executing
- Inspect an early result. Update the task and project notes as you learn

Adding “think carefully” cannot supply your sample definition or tell the agent which tradeoffs you would make.

Weidmann, Xu, and Deming (2025) find that performance leading AI agents strongly predicts performance leading human teams in a lab study.

Decisions that you skip, the agent makes silently

The same ownership problem appears in data analysis.

Request: “Estimate abnormal returns after initial public offerings (IPOs). Report a figure and a table.”

Decisions left to the agent:

- Buy-and-hold or cumulative abnormal returns
- The benchmark
- The event window
- The treatment of delisted firms
- How to match firms across datasets

Exercise: find the decisions that a vague request leaves to the agent

> Use the Current Population Survey (CPS) to estimate whether state minimum wage increases reduced teen employment.

1. What potentially important decisions are left unspecified?
2. How would you provide more structure in the prompt?

Small tasks with specific feedback are easier to check

- One goal per session: “Get the cleaning script to run”
- Review each result before the next step
- Stop the agent early if it goes the wrong way

Vague feedback	Specific feedback
“This seems off.”	“Find why four county reports are missing from the download manifest.”

Ensure reproducibility: ask for a script

Request	Result
“Download the election reports.”	PDFs, with no record of how
“Write `code/download_reports.py` and run it.”	PDFs and a procedure that you can run again

Do not outsource your understanding

- An agent can propose a design or summarize papers quickly
- You must understand the choices, evidence, and disagreements behind your project
- Read the important papers and explain your contribution in your own words

The individual final exam tests that understanding without the agent.

Example: AI-assisted literature review

AI can search, collect, summarize, and suggest how to situate your paper.
You decide which claims survive your own reading.

Step	What to do	What to save
Find	Start with Connected Papers. Script searches the Semantic Scholar API for papers, citations, and recommendations	`candidates.csv` with IDs, queries, and dates
Explore	Agent collects available full text, summarizes papers, and proposes how they relate to yours	Claim table with paper and page references
Own	Read central papers and their literature reviews. Check claims and write your synthesis	Your account of the field and your contribution

The agent's proposed map is a **reading plan and a set of claims to check**. It cannot build your expertise for you.

Takeaways: intellectual ownership

- The agent does the work. You own the decisions and the result.
- Make the important decisions before the agent makes them for you
- Write and revise your plan. Use the agent to test feasibility, then work in small steps
- Ask for scripts, not results
- The individual final exam checks what you can explain without an agent

4. Verification

- Delegate more where checking is cheap
- Build verification checks as you work, not at the end
- Choose the right kind of check for each risk

First ask: how hard is the result to check?

Implementation cost: effort to produce a result.
Verification cost: effort to tell whether it is correct.

	Cheap to make	**Costly to make**
Costly to verify	Caution Easy output, hard audit	Trap Tempting to delegate, hard to audit
Cheap to verify	Low stakes Easy to redo or check	Delegate Large effort, clear check

Delegate most when verification is cheap. Make verification easier before delegating a task in the upper row.

Exercise: place each task in the grid

	Cheap to make	**Costly to make**
Costly to verify	Caution	Trap
Cheap to verify	Low stakes	Delegate

1. Change the colors of a figure
2. Implement an estimator from a 2023 paper that has a Stata package
3. Extract one section from 120 annual reports
4. Summarize the literature on minimum wages, with citations
5. Build a 20-step cleaning pipeline for a survey

What could make a hard-to-check task easier to verify?

Unchecked output accumulates as verification debt

- Each result that you accept without checking adds to the debt
- More autonomy moves all the checking to the end
- A polished result gives no information about whether it is correct

Next: make one consequential choice visible as the sample is built.

Example: track sample losses

Bad record after a merge: “10,000 election rows before. 9,908 after.” It hides which 92 rows disappeared and why.

Better: Save a report from the merge code and make its counts reconcile.

```
Candidate rows before merge        10,000
Matched to county lookup            9,908
Unmatched: county field blank          57
Unmatched: county not in lookup        35
```

```
assert len(matched) + len(unmatched) == len(input_rows)
assert unmatched["reason"].notna().all()
```

Commits let you review one change at a time

```
$ git log --oneline
a41c9e2 Audit county vote totals
7d02b18 Parse candidate rows from PDFs
3f9a6c1 Test three report layouts
c18e4d0 Save the report URL manifest
09b2f77 Create project structure
```

Four kinds of checks catch different errors

Check	What it compares	Use it when
Unit test	A function’s output with a known answer on a small input	Function logic can go wrong
Assertion	Real data with rules that must hold	A pipeline step can lose or duplicate rows
Regression test	New output with saved earlier output	A code change should preserve results
External check	Your result with an independent source	A published number or official series exists

A unit test checks logic on a small known input

```python
# tests/test_merge.py
import pandas as pd
from code.clean import merge_policy

def test_merge_keeps_one_row_per_person():
    people = pd.DataFrame({"id": [1, 2, 3], "state": ["MA", "MA", "NH"]})
    policy = pd.DataFrame({"state": ["MA", "NH"], "min_wage": [15.00, 7.25]})

    merged = merge_policy(people, policy)

    assert len(merged) == 3
    assert merged["id"].is_unique
    assert merged.loc[merged["id"] == 3, "min_wage"].item() == 7.25
```

An assertion stops the pipeline on a broken assumption

```
n_people = len(people)

merged = people.merge(
    policy, on=["state", "year"], how="left", validate="many_to_one"
)

assert len(merged) == n_people, "Merge changed the number of people"
assert merged["min_wage"].notna().mean() > 0.99, "Too many unmatched states"
assert merged["age"].between(16, 64).all(), "Age outside sample range"
```

External checks use a source that your code did not make

- Reproduce a published number before you extend it
- Compare your totals with an official series
- Compare your estimator with the authors' implementation
- Check a random sample of records by hand

Exercise: choose the check for each risk

1. A function converts weekly earnings to hourly wages, with top codes
2. Next year’s survey file may add a new code for “refused”
3. The agent rewrites the cleaning script to run faster
4. You construct county unemployment rates from microdata
5. A model extracts firm names from 2,000 news articles

Unit test, assertion, regression test, external check, or hand inspection?

Look at the data to learn what to check

What someone saw	Check that it motivated
8 of 120 filings had no extracted section	An extraction report for each filing
Los Angeles County had several rows per year	One row per county-year
The fintech share dropped in 2018	Flag the year when lender IDs changed
Loan amounts cluster at round numbers	Separate round-number effects from the policy effect

Exercise: evaluate the agent's fix for a duplicate row

```
people.csv          10,000 rows   one row per person
state_policy.csv        52 rows   expected: one row per state
merged.csv          10,212 rows
```

The agent's fix: keep the first row for each state. All tests pass.

Question: What is wrong with this fix?

An agent can pass tests without correct code

If the goal is “the tests pass,” an agent can:

- Edit the test instead of the code
- Hard-code the expected value
- Drop the rows that fail

Passing tests means only that the code satisfies **those tests**. Read the tests, inspect source records, and review any diff that changes a test.

Make final paper numbers traceable to code

- While drafting in Google Docs, mark citations and generated numbers with `[XXX placeholder]`
- The final paper will be written in a script, such as Quarto or LaTeX. Its figures, tables, and numbers in text will come from files in `output/`
- Trace each final number to an output file, a script, and a commit

Takeaways: verification

- Delegate the most where checking is cheap. Make checking cheap where it is not.
- Build checks as you work. Keep commits, decisions, and attrition tables as you go.
- Confirm the agent's claims against the code and the output
- Match each risk to a check: unit, assertion, regression, external, or by hand
- You decide what the tests check, and review any changes to tests.

5. Permissions

- Limit folder access and tool access separately
- Design the environment, then choose the approval mode
- Sandboxes do not keep data from the provider

Limit folder access and tool access separately

Set a boundary for	Ask
Files and folders	What can the agent read or change?
Tools	Can it run shell commands, Python, or Git?
Network	Can it reach websites or send data to an API?

Starting in `election-project/` sets the **working directory**. It may not limit what the agent can read elsewhere.

Check effective permissions before the first task

1. Start the agent in `election-project/`.
2. Keep API keys and restricted data outside its reach.
3. Inspect its effective file, tool, and network permissions.

Project defaults can live in tool settings: Codex `.codex/config.toml` or Claude Code `.claude/settings.json`. `AGENTS.md` cannot enforce them.

The approval mode sets when the agent asks to act

Mode	The agent asks before	Risk depends on
Manual	Most actions	Whether you read each request
Auto	Only actions that a classifier or rule flags	The classifier and the rules
Bypass (“YOLO”)	Nothing	The environment

Check the mode shown by your tool. Defaults vary by version, interface, model, and settings. None of these modes checks whether the extracted data are right.

Auto mode does not verify extracted data

Phase	Let the agent do	Researcher check before proceeding
Plan	Inspect three PDF types and propose a schema	Approve row definition and missing-value rules
Pilot	Write and run code on a small, varied batch	Compare source pages with output rows by hand
Full run	Process saved PDFs, log failures, make an audit report	Check coverage, totals, and a fresh random sample

Auto mode can reduce prompts during a bounded task. Pause at these stage boundaries even when the agent can run without asking.

Approving without reading gives no protection

- After many prompts, most people approve without reading
- Then each prompt costs time and protects nothing
- Real protection comes from limits that the agent cannot change

Ask what the environment allows, not whether you trust the model

1. Which folders can the agent see?
2. Can it write, install, or delete?
3. Are keys, passwords, or tokens within reach?
4. Can it reach the internet?
5. What is the worst case, and can you undo it?

Recap aside: What is a virtual environment?

- A Python virtual environment is a project-specific set of installed packages
- Create it in the project, activate it, and install the packages the project needs
- Record those package requirements so someone else can recreate the setup

A virtual environment does not isolate the agent

	Virtual environment	Sandbox or container
Purpose	Keep project packages separate	Limit what the agent can reach
Can limit file reads?	No	Yes
Can limit network?	No	Yes

A built-in sandbox controls what tools can reach

Tool	Default	Writes	Reads	Network
Claude Code	Off. Turn on with `/sandbox`	Project folder	Broad unless you add deny rules	Asks for each new domain
Codex CLI	On	Project folder. `.git` is read-only.	Full disk	Off
Gemini CLI	Off. Turn on with `--sandbox`	Project folder	Broad in the default macOS profile	On in the default macOS profile

Advanced option: put the agent in a container

- A **sandbox** restricts tool actions, such as writes or network access. Read limits depend on its configuration.
- A **container** packages the project's software environment and exposes selected folders through mounts. This helps teammates run the same code.
- Configure mounts and network access to limit what the agent can reach. Save outputs in a mounted folder so they remain available on your computer.

For a simple course project, start with Git, a virtual environment, and careful tool permissions. Add a container when isolation or portability justifies the setup.

Exercise: what does this setup protect?

You store an API key **outside** `election-project/`. You start the coding agent inside the project with a Python virtual environment active. It can run Python commands.

Question: What problem has this setup solved, and what access risk remains?

Git reminder: Never commit API keys or SSH keys. Keeping the key outside the repository helps prevent a commit, but Git rules do not limit what an agent can read.

A sandbox does not keep data from the provider

- A sandbox limits what the agent can reach
- Each file that the agent reads is still sent to the provider
- Data-use terms differ by plan and provider
- Course rule: commercial subscriptions only for low-risk data

Third-party skills run someone else's instructions

- A web page or file can also contain instructions that the agent follows
- Read a third-party skill before you install it
- Prefer official or well-reviewed sources

Recommended default for this course

1. Commit so that the working tree is clean
2. Create a branch for the task
3. Start the agent in the project folder, never in your home folder
4. Keep secrets and restricted data out of that folder
5. Use manual or auto approval
6. Review the diff before you merge

Exercise: rank four agent setups by risk

1. Auto mode, clean branch, project folder with a `.env` file that holds your WRDS password: “Fix the download script.”
2. Bypass mode in a container with only the project mounted, network on: “Scrape 5,000 county web pages overnight.”
3. Manual mode, a folder with a friend’s medical survey data: “Summarize the data.”
4. Bypass mode in a container with only the project mounted, no network: “Run all 200 specifications overnight.”

For each, what one change reduces the risk most?

Terminology recap: permissions

Mode	Claude Code	Codex CLI	Gemini CLI
Ask before edits and commands	Manual (`default`)	`--sandbox read-only`	`default`
Allow edits without asking	`acceptEdits`	Auto preset: `workspace-write`, `on-request`	`auto_edit`
A model reviews risky actions	`auto`	`approvals_reviewer = "auto_review"`	
Read and plan only	`plan`	`/plan`	`plan`
Never ask	`--dangerously-skip-permissions`	`--yolo`	`--approval-mode=yolo`

Takeaways: permissions

- Decide folder access and tool access separately
- Design the environment first. Then choose the approval mode.
- A virtual environment isolates packages, not the agent
- Default sandboxes limit writes more than reads
- The provider receives everything that the agent reads

Summary: five ideas for research with agents

Section	Main idea
Agentic AI	An agent is a model with tools, working in a loop in your project
Context management	Files persist and sessions do not, so codify what you repeat
Intellectual ownership	You make the decisions, and you are accountable for the result
Verification	Build checks as you work, and confirm every claim
Permissions	Limit the environment, then give the agent freedom inside it

References

Tool documentation

- Claude Code
- Codex
- Gemini CLI
- Semantic Scholar API

Research tools and papers

- Connected Papers
- Weidmann, Xu, and Deming, *Measuring Human Leadership Skills with AI Agents*

Agent research and examples

- Paul Goldsmith-Pinkham, agent series and NBER talk materials
- Garicano, Li, and Wu, *Messy Jobs*
- HumanLayer, "Advanced context engineering for coding agents"